REPORT

Reviewer report: pyinvoke/invoke#1099: Use explicit

re-exports for mypy strict mode

Run invoke-1099, tested 2026-11-09 13:09 IST by ops/daily_run.py (route source-pair). Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 15:05 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

NEEDS ATTENTION

Recommended action: Reject Confidence: High

The run is trustworthy and found a difference a person should look at. Every check passed and every claim cites

a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Use explicit re-exports for mypy strict mode (pyinvoke/invoke#1099).

• What we observed: probe: 1 observation(s) need attention: c4.test_file_exists does not differ although the claim

says it does (C4); probe model-probe-public-exports has no measured line coverage (no coverage data); it does

not count as covering the change; tests: 32 test(s) fail on main and pass on the PR; 204 test(s) ran on main, 204

on the PR; 0 new test(s) pass, 165 unchanged (1 of them skipped on both sides); 8 test failure(s) pre-existing

and identical on both sides.

• What you should do: Do not approve as is. Read 'What changed in behavior' and decide with the PR author.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 3.5 GB (of 15.6 GB).

• Setup check: free disk 92.5 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: invoke python setup install instructions https://github.com/pyinvoke/invoke

site:github.com/pyinvoke/invoke

• Query: invoke python documentation install https://github.com/pyinvoke/invoke site:github.com/pyinvoke/invoke

• Query: invoke python install requirements https://pypi.org/project/invoke/ https://github.com/pyinvoke/invoke

site:pypi.org

• Source: invoke-common-tasks · PyPI — https://pypi.org/project/invoke-common-tasks/

• Source: invoke · PyPI — https://pypi.org/project/invoke/

• Source: invoke/invoke/program.py at main · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/blob/master/invoke/program.py

• Source: invoke/invoke/runners.py at main · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/blob/master/invoke/runners.py

• Source: ImportError: cannot import name yaml2 · Issue #319 · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/issues/319

• Source: Invoke 0.13 breaks on Windows (unavailable default shell) · Issue #371 · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/issues/371

• Source: Dealing with virtual environments · Issue #838 · pyinvoke/invoke · GitHub —

https://github.com/pyinvoke/invoke/issues/838

• Setup plan: the same packages on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.9

• Installed icecream 2.2.0 from project (named by the error and listed in the project) on main and pr.

• Before rerun: could_not_verify: tests produced no results on both sides (collection failed; see pytest logs);

probe: 1 observation(s) shown as advisory only (claim not shown on these inputs; a related change was proven):

c4.test_file_exists (C4); probe: claim(s) C1, C2, C3, C5 proven

• After rerun: needs_attention: probe: 1 observation(s) need attention: c4.test_file_exists does not differ although

the claim says it does (C4); probe model-probe-public-exports has no measured line coverage (no coverage

data); it does not count as covering the change; tests: 32 test(s) fail on main and pass on the PR; 204 test(s) ran

on main, 204 on the PR; 0 new test(s) pass, 165 unchanged (1 of them skipped on both sides); 8 test failure(s)

pre-existing and identical on both sides

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (173 unchanged, 3 runs per side) | pytest-pr-1.log:443 «196 passed, 1 skipped, 5

warnings, 9 errors in 1.19s»

| P-C1 | After applying the PR, running mypy --no-implicit-reexport on from invoke import task passes type

checking. (PR description) | proven | probe model-probe-public-exports, c1.invoke_has_all: value: false -> true |

probe-model-probe-public-exports-pr-1.jsonl:1 «"observation": "c1.invoke_has_all"»

| P-C2 | After applying the PR, from invoke import * does not leak internal names such as metadata. (PR

description) | proven | probe model-probe-public-exports, c2.star_import_leaks_metadata: value: true -> false |

probe-model-probe-public-exports-pr-1.jsonl:3 «"observation": "c2.star_import_leaks_metadata"»

| P-C3 | After applying the PR, both invoke/__init__.py and invoke/parser/__init__.py define an __all__ variable

listing the public names. (PR description) | proven | probe model-probe-public-exports, c3.invoke_has_all: value:

false -> true | probe-model-probe-public-exports-pr-1.jsonl:4 «"observation": "c3.invoke_has_all"»

| P-C4 | The new test file tests/test_public_exports.py contains tests that fail on the original code and pass with

this change. (PR description) | not observed | probe model-probe-public-exports, c4.test_file_exists: same on

both sides: false | probe-model-probe-public-exports-pr-1.jsonl:6 «"observation": "c4.test_file_exists"»

| P-C5 | Applying the PR does not introduce any new test failures; the number of failing tests remains the same

as in the original code (114 pre■existing failures). (PR description) | proven | probe model-probe-public-exports,

c5.invoke_imports_without_error: same on both sides: true | probe-model-probe-public-exports-pr-1.jsonl:7

«"observation": "c5.invoke_imports_without_error"»

Citation check (PASS): 6 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 204 / 204 | | Tests unchanged | 173 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 32 |

tests.test_public_exports::test_invoke_defines_explicit_export_list,

tests.test_public_exports::test_invoke_export_list_includes_expected_name[Argument],

tests.test_public_exports::test_invoke_export_list_includes_expected_name[Call],

tests.test_public_exports::test_invoke_export_list_includes_expected_name[CollectionNotFound],

tests.test_public_exports::test_invoke_export_list_includes_expected_name[Collection]

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 8 | tests._support.depth_first::build,

tests._support.depth_first::clean, tests._support.depth_first::clean_html, tests._support.depth_first::clean_tgz,

tests._support.depth_first::deploy

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe-public-exports (model (nvidia/nemotron-3-super-120b-a12b, live),

model-probes/model-probe-public-exports.py, frozen 10501b6c105f at 2026-11-09T07:41:05.057107+00:00):

Check public exports and star import behavior

• P-C1 (PR description): After applying the PR, running mypy --no-implicit-reexport on from invoke import task

passes type checking. Expects the covered observations to differ.

• P-C2 (PR description): After applying the PR, from invoke import * does not leak internal names such as

metadata. Expects the covered observations to differ.

• P-C3 (PR description): After applying the PR, both invoke/__init__.py and invoke/parser/__init__.py define an

__all__ variable listing the public names. Expects the covered observations to differ.

• P-C4 (PR description): The new test file tests/test_public_exports.py contains tests that fail on the original code

and pass with this change. Expects the covered observations to differ.

• P-C5 (PR description): Applying the PR does not introduce any new test failures; the number of failing tests

remains the same as in the original code (114 pre■existing failures). Expects the covered observations to stay

the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.invoke_has_all | P-C1 | as claimed | value: false -> true | probe-model-probe-public-exports-main-1.jsonl:1

«"observation": "c1.invoke_has_all"»
probe-model-probe-public-exports-pr-1.jsonl:1 «"observation":

"c1.invoke_has_all"»

| c1.parser_has_all | P-C1 | as claimed | value: false -> true | probe-model-probe-public-exports-main-1.jsonl:2

«"observation": "c1.parser_has_all"»
probe-model-probe-public-exports-pr-1.jsonl:2 «"observation":

"c1.parser_has_all"»

| c2.star_import_leaks_metadata | P-C2 | as claimed | value: true -> false |

probe-model-probe-public-exports-main-1.jsonl:3 «"observation":

"c2.star_import_leaks_metadata"»
probe-model-probe-public-exports-pr-1.jsonl:3 «"observation":

"c2.star_import_leaks_metadata"»

| c3.invoke_has_all | P-C3 | as claimed | value: false -> true | probe-model-probe-public-exports-main-1.jsonl:4

«"observation": "c3.invoke_has_all"»
probe-model-probe-public-exports-pr-1.jsonl:4 «"observation":

"c3.invoke_has_all"»

| c3.parser_has_all | P-C3 | as claimed | value: false -> true | probe-model-probe-public-exports-main-1.jsonl:5

«"observation": "c3.parser_has_all"»
probe-model-probe-public-exports-pr-1.jsonl:5 «"observation":

"c3.parser_has_all"»

| c4.test_file_exists | P-C4 | no change although the claim says it changes | same on both sides: false |

probe-model-probe-public-exports-main-1.jsonl:6 «"observation":

"c4.test_file_exists"»
probe-model-probe-public-exports-pr-1.jsonl:6 «"observation": "c4.test_file_exists"»

| c5.invoke_imports_without_error | P-C5 | as claimed | same on both sides: true |

probe-model-probe-public-exports-main-1.jsonl:7 «"observation":

"c5.invoke_imports_without_error"»
probe-model-probe-public-exports-pr-1.jsonl:7 «"observation":

"c5.invoke_imports_without_error"»

Citation check (PASS): 14 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (3.7 s run).

Coverage gate (python, confidence normal): 19 of 19 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| invoke/__init__.py | 13 | 13

| invoke/parser/__init__.py | 6 | 6

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 06f3320e772b of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: pytest exit 1; 8 failure(s)/error(s) pre-existing and identical on both sides.

• Changed code was exercised (coverage gate): 19 of 19 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 6 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 14 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Behavior difference found | finding | probe: 1 observation(s) need attention: c4.test_file_exists does not differ

although the claim says it does (C4); probe model-probe-public-exports has no measured line coverage (no

coverage data); it does not count as covering the change; tests: 32 test(s) fail on main and pass on the PR; 204

test(s) ran on main, 204 on the PR; 0 new test(s) pass, 165 unchanged (1 of them skipped on both sides); 8 test

failure(s) pre-existing and identical on both sides | This is about the PR, not the run: read the failing tests and

their log lines, then decide.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | pytest exit 1; 8 failure(s)/error(s) pre-existing and identical on both sides

| Changed code was exercised (coverage gate) | PASS | 19 of 19 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 6 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 14 citation(s), every one re-read from its log file and

matched

Checklist

- I read the verdict and the plain-English summary.
- Every 'false' or 'unmentioned change' row has a log line I can open.
- Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).
- I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide REPORT=poc-main/reports/daily/2026-11-09/invoke-1099 DECISION=approve (or reject, approve_with_notes, re_run).

Rerun command
make check PR=https://github.com/pyinvoke/invoke/pull/1099

Evidence links
• Pull request pyinvoke/invoke#1099
• record.json
• pr.diff
• setup-main.log
• setup-pr.log
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-probe-public-exports-main-1.jsonl
• probe-model-probe-public-exports-pr-1.jsonl
• main run 1 result: pytest-main-1.log:411 «164 passed, 1 skipped, 5 warnings, 9 errors in 1.00s»
• pr run 1 result: pytest-pr-1.log:443 «196 passed, 1 skipped, 5 warnings, 9 errors in 1.19s»

Glossary
| Term | Meaning
| main | The library as it is today, without the PR.
| PR | Pull request: a proposed change to the library.
| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.
| flaky | A test that gives different results when nothing changed; it is ignored as evidence.
| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 18:39 IST